ELSEVIER

Simulation Practice and Theory 9 (2001) 73–90

SIMULATION PRACTICE AND THEORY

www.elsevier.com/locate/simpra

Distributed web-based simulation experiments for optimization ☆

Enver Yücesan [a], Yuh-Chuyn Luo [b], Chun-Hung Chen [c,*], Insup Lee [d]

[a] *Technology Management Area, INSEAD, Boulevard de Constance, 77305 Fontainebleau Cedex, France*

[b] *Department of Computer and Information Science, Chung-Chen Institute of Technology, Taiwan, ROC*

[c] *Department of Systems Engineering and Operations Research, George Mason University, MS 4A6, 4400 University Drive, Fairfax, VA 22030, USA*

[d] *Department of Computer and Information Science, University of Pennsylvania, Philadelphia, PA 19104, USA*

Received 19 July 2000; received in revised form 12 May 2001

Abstract

The web has grown tremendously over the past decade – with applications in education, marketing, financial services, and supply chain management. Web technology also has a significant impact on computer simulation. Most of the effort in web-based simulation is aimed at modeling, particularly at building simulation languages and at creating model libraries that can be assembled and executed over the web. This paper focuses on efficiency of simulation experimentation for optimization. We introduce a framework for combining the statistical efficiency of simulation optimization techniques with the effectiveness of parallel execution algorithms. In particular, a novel simulation sampling procedure, the Optimal Computing Budget Allocation (OCBA) algorithm, is implemented in a web-based environment for low-cost parallel and distributed simulation experimentation. A prototype implementation with some experimental results is presented to show the viability of web-enabled simulation environments. © 2001 Elsevier Science B.V. All rights reserved.

Keywords: Simulation optimization; Ranking and selection; Java programming

☆ This work has been supported in part by INSEAD Research through grant #2010248, NSF under Grants DMI-9732173, DMI-0002900, DMI-0049062, by Sandia National Laboratories under Contract BD-0618, and by George Mason University Research Foundation.

* Corresponding author. Tel.: +703-993-3572; fax: +703-993-1521.

E-mail addresses: enver.yucesan@insead.fr (E. Yücesan), yuchun@ccit.edu.tw (Y.-C. Luo), cchen9@gmu.edu (C.-H. Chen), lee@ithan.cis.upenn.edu (I. Lee).

0928-4869/01/$ - see front matter © 2001 Elsevier Science B.V. All rights reserved.
PII: S0928-4869(01)00037-4

1. Introduction

The web has experienced tremendous growth since its introduction in early 1990s [33]. While web-based applications for personal entertainment have been most visible, the web is having a drastic impact on many industries. Examples of innovative applications include distance learning applications for medicine, military and technical training, on-line trading in financial markets, new marketing channels, new forms of supplier relationships, and new forms of logistics activities. The web also has a growing impact on computer simulation. Discrete-event simulation is a popular tool for designing large man-made systems such as communication networks, traffic systems, and manufacturing facilities since no reliable closed-form analytical models exist for such systems. In addition to excellent graphical animation capability, simulation offers virtually unlimited modeling flexibility as arbitrary levels of system detail can be incorporated into a simulation model.

However, the effectiveness of computer simulation experiments is increasingly strained by increased levels of detail in simulation models (e.g., explicit representation of an increasing number of resources – such as people, machines, tools, and jobs in manufacturing models), considerably higher levels of activity in the modeled system (e.g., millions of messages to be handled by a telecommunications network), and significantly improved levels of reliability of modeled systems (e.g., a highly reliable airline ticket reservation computer). In those cases the execution speed of models does not improve, in spite of the availability of faster hardware platforms. In cases with rare events, significantly longer simulation runs are required to obtain reliable performance estimates.

Efficiency becomes an even greater concern when simulation is used for decision-making. In this setting, simulation must be performed repeatedly for comparing many design alternatives. Furthermore, to obtain a good statistical estimate for a design decision, a large number of simulation replications (or samples) are usually required for each design alternative. If the number of design alternatives is large, the total simulation cost could be prohibitively high. Various schemes have been proposed to enhance the effectiveness of simulation experiments. On the modeling side, alternative modeling approaches have been introduced to reduce the inherent complexity of models either by implicitly representing most system entities [37] or by reducing the size of the state space of the underlying model [22,23]. On the analysis side, various approaches have been introduced to improve the statistical efficiency of simulation experiments [7,36]. Finally, on the execution side, algorithms have been developed for parallel execution [6,16,29].

The objective of this paper is to introduce a framework for combining the statistical efficiency of simulation optimization techniques with the effectiveness of parallel execution algorithms. In particular, a novel simulation sampling procedure, which we call the Optimal Computing Budget Allocation (OCBA) algorithm, is implemented in a web-based environment for low-cost parallel and distributed simulation experimentation.

Our work focuses on the parallel execution of a simulation experiment for optimization. However, instead of equally distributing simulation replications for different

design alternatives, OCBA offers an intelligent way to allocate simulation experiments to processors. More specifically, OCBA is a novel scheme aimed at identifying the optimal system through an optimal experimental design. The underlying ideas are as follows. Intuitively, to ensure the correct selection of the optimal design, a larger portion of the computing budget should be allocated to those designs that are critical in identifying the best design. That is, a larger number of replications must be conducted with those critical designs to reduce estimator variance. On the other hand, limited computational effort should be expanded on non-critical designs that have little effect on identifying the best design – even if they have large variances. In doing so, less computational effort is spent on simulating non-critical designs and more computational effort is spent on simulating critical designs; hence, the overall simulation efficiency is improved. Ideally, we want to optimally choose the number of simulation replications for all designs to maximize simulation efficiency with a given computing budget. In fact, this question is equivalent to optimally deciding which designs will receive additional computing budget for continuing the simulation further or to find an optimal way to reach an optimal design.

Our objective in designing and implementing a web-based simulation optimization system is to emphasize the power of web technologies for the experimental design and output analysis phases of a simulation study. Our immediate concern is thus capability (or functionality) rather than run-time performance. Issues of portability, maintainability, and conformance to standards are crucial in demonstrating the feasibility of web-enabled simulation environments. We will subsequently focus on execution speed through the design of intelligent distribution algorithms, and exploit emerging technologies aimed at speeding up communication over the Internet.

The paper is organized as follows. Section 2 provides the necessary background on simulation optimization (and, in particular, on OCBA), on parallel and distributed simulation, and on web-based simulations. Section 3 describes a single-processor implementation of web-based OCBA, together with some performance assessment. The prototype for distributed simulation over the Internet is introduced in Section 4. Current research efforts are described in Section 5.

2. Background

2.1. Simulation optimization

Simulation enables the comparison of various design alternatives before implementing any of the required physical changes. Suppose we want to compare a given number, say k, different systems (competing designs or alternative operating policies). We conduct N simulation replications (terminating simulation) or N simulation samples (steady-state simulation) for each of the k designs. Therefore, we need kN simulation replications (samples). For ease of discussion, in this paper, we will focus on terminating simulation and so N is the number of simulation replications. Simulation results become more accurate as N increases. However, the ultimate accuracy (typically expressed as a confidence interval) of the performance measure estimation

for each design cannot improve faster than $O(1/\sqrt{N})$, the result of averaging i.i.d. noise. To obtain an acceptable statistical estimate for a design, a large N is usually required for each design alternative, implying a long computation time. If the accuracy requirement is high (N is not small) and if the total number of competing designs is large (k is large), then kN can be very large. This may easily make total simulation cost prohibitively high and preclude the feasibility of correct simulation optimization.

The effective reduction of computation costs while obtaining a good decision is therefore crucial. Outside simulation, Dudewicz and Dalal [12] propose a two-stage procedure for selecting either the best design or a design that is very close to the best system. In the first stage, all systems are simulated with a fixed number of replications. Based on the results of the first stage, the number of additional simulation replications for each design in the second stage is estimated in order to reach the desired confidence level. Rinott [34] presents an alternative way to estimate the required number of simulation replications in the second stage. Many researchers have extended this idea to more general ranking and selection settings, in conjunction with new developments. Chiu [9], Gupta and Panchapakesan [18], Bechhofer et al. [1], and Hsu [21] present methods based on the classical statistical model adopting a frequentist view. On the other hand, a Bayesian framework for constructing ranking and selection procedures is used by Berger [2], Berger and Deely [3], Bernardo and Smith [4], Gupta and Berger [19], and, in simulation, by Chick [8].

Chen et al. [7] formulate the procedure of selecting the best design as an optimization problem, which optimally allocates a computing budget and processors to the designs under evaluation. They present a solution technique to this budget allocation problem. Preliminary tests indicate that the proposed technique is significantly faster than the traditional two-stage procedures of Dudewicz and Dalal [12] and Rinott [34].

Let SQ denote the overall simulation quality; examples of SQ include the mean squared error of the performance measure or the probability of correctly selecting the truly best design. Also denote by N_i the number of simulation replications for design i ($i = 1, \ldots, k$). If the simulation experiment is executed sequentially on a single processor, the budget allocation problem can be expressed as

$$\begin{aligned} \max_{N_1,\ldots,N_k} \quad & \mathrm{SQ} \\ \text{s.t.} \quad & N_1 + N_2 + \cdots + N_k = B, \end{aligned}$$

where B is the given computing budget. Intuitively, this technique provides an optimal way to reach an optimal design using simulation experiments. As a greedy solution to the above OCBA problem (i.e., to find the best $N_1, N_2, \ldots, N_k$), we sequentially select a subset of "promising" designs in each iteration of the computational experiment. This procedure continues until the computing budget B is exhausted. We define *promising designs* as those maximizing the estimated improvement of SQ (or ESQ). A critical issue in this approach is the determination of a set of promising designs, or more specifically, the estimation of the new ESQ if additional τ testing samples are performed on design s.

Let $\mathrm{ESQ}(N_1, N_2, \ldots, N_{s-1}, N_s + \tau, N_{s+1}, \ldots, N_k)$ denote an *estimated SQ* if additional τ simulation replications *were* performed on design s. ESQ is computed using the statistical information after $N_1, N_2, \ldots, N_k$ simulation replications are completed for designs $1, \ldots, k$, respectively. Based on a Bayesian model, Chen et al. [7] present an effective way to estimate ESQ, using the information of simulated data, if SQ is defined as the probability of correctly selecting the true best design $(P\{CS\})$. This scheme will be adopted in this paper to implement the efficient simulation experiment planner.

To maximize the level of confidence in selecting the best design, in each iteration, we further simulate the designs that have a maximum Promising Index (PI) defined as follows:

$$\begin{aligned}\mathrm{PI}(s) \equiv\ & \mathrm{ESQ}(N_1, N_2, \ldots, N_{s-1}, N_s + \tau, N_{s+1}, \ldots, N_k) \\ & - \mathrm{SQ}(N_1, N_2, \ldots, N_{s-1}, N_s, N_{s+1}, \ldots, N_k).\end{aligned}$$

Under the above framework, promising refers to high improvement of the overall comparison confidence level. Since we intend to maximize the level of confidence for selecting the best design, we select and simulate a subset of most promising designs in each iteration; next we repeat the process until the total computing budget B is exhausted.

Using the concept of optimal computing budget allocation we develop an iterative experimentation procedure for selecting the optimal design. The procedure is first summarized as follows. We assume that at the beginning of the experiment there is no knowledge about PI or SQ of any of the designs, nor is there any other basis for allocating the computing budget. Therefore, we first simulate all designs with n_0 replications. A PI is then calculated for all designs, and a subset of m designs is selected for further simulation based on their PI's, where $m \leqslant k$. We have $m = 1$ when we consider the case of one computation processor. m is greater than 1 when we have multiple processors. The procedure is outlined in the following pseudo-code format:

A sequential algorithm for Optimal Computing Budget Allocation (*OCBA*)

Step 0. Perform n_0 simulation replications for all designs; $l \leftarrow 0$; $N_1^l = N_2^l = \cdots = N_k^l = n_0$, $B = B - kn_0$.

Step 1. If $B = 0$, then stop, else, go to Step 2.

Step 2. Calculate $\mathrm{PI}(s)$ for all designs $s = 1, 2, \ldots, k$.

Step 3. Find the set $S(m) \equiv \{s : \mathrm{PI}(s)$ is among the highest $m\}$, where m is the number of threads or parallel processors available.

Step 4. Perform additional τ simulation replications for design i, $i \in S(m)$. Set $N_i^{l+1} \leftarrow N_i^l + \tau$ for $i \in S(m)$, and $N_i^{l+1} \leftarrow N_i^l$ for $i \notin S(m)$, $B \leftarrow B - m\tau$, $l \leftarrow l + 1$, go to Step 1.

2.2. Parallel and distributed simulation

We distinguish the following four approaches to exploit parallelism in discrete-event simulation to reduce computation time.

- *Dedicated execution*: Dedicated functional units execute specific sequential simulation functions such as random number generation, future events list management, and statistics collection [10].
- *Hierarchical decomposition*: The simulation model is decomposed into a hierarchy so as to allow an event consisting of several sub-events to be processed concurrently [11].
- *Parallel replication*: Several replications of a sequential simulation are executed independently on different processors [20].
- *Parallel execution*: Simulation model is executed on a parallel computer by decomposing the simulation model implementation into a set of concurrently executing processes [29].

Dedicated execution and hierarchical decomposition exploit the cost advantage of microcomputers and high-bandwidth lines by partitioning the simulation problem and executing the parts in parallel. There are, however, inherent drawbacks in these two approaches. In dedicated execution, the speedup tends to be very limited, as the management of the future-events list becomes the bottleneck.

The performance improvement in hierarchical decomposition is largely model-dependent; for example, some queueing network models have been successfully simulated in a distributed fashion by simulating each queueing node on a separate processor. For other models, however, decomposing the model for parallel execution is not trivial. Som and Sargent [35] propose a conceptual algorithm for hierarchical decomposition. Lin [26] introduces algorithms for critical path analysis aimed at uncovering inherent parallelism in simulation models.

In parallel replication, each processor must contain sufficient memory to hold the entire simulation model. This is becoming less of a constraint with the rapid advances in hardware. Nevertheless, one important problem remains: if one desires to distribute the experiment such that the results of one replication are used to determine which run to execute next, then the execution of the overall experiment becomes once again sequential.

Finally, in parallel execution, two research and development branches have emerged: Parallel and Distributed Simulation (PADS) and Distributed Interactive Simulation (DIS) [16]. PADS is concerned with the fast execution of a single large simulation model, where speedup relative to a sequential implementation is the main performance metric. In PADS, a large-scale model is developed as a single integrated system within one modeling environment. DIS, on the other hand, interconnects several autonomous simulators in a distributed environment typically for training exercises (e.g., battlefield simulations). Realism of the virtual environment and scalability are the main priorities in DIS.

The application of parallel simulation technology has been limited; the simulation community has largely been reluctant to explore the potential gains offered by this technology. In fact, this lack of acceptance has triggered a heated discussion about the future of parallel discrete-event simulation [17]. We believe that Internet and web-based technologies provide a viable infrastructure for parallel discrete-event simulation. Internet eliminates the need for expensive parallel hardware, while the

Internet Protocol (IP) unifies diverse networking technologies and administrative domains. In other words, IP ensures uniform connectivity without requiring uniform behavior. The Internet can therefore be viewed as an appealing infrastructure for distributed simulation [13].

2.3. Web-based simulation

Web-based simulation represents the convergence of simulation methodology and World Wide Web technologies. The key enabler of web-based simulation is the *Java* language introduced by *Sun Microsystems*. Java is an object-oriented (OO) programming language for the web, which therefore supports such OO features as classes, encapsulation, polymorphism, and inheritance. Furthermore, Java aims for universal portability; Java source code is compiled into byte code and browsers provide a byte code interpreter to execute the code produced by a Java compiler. On the one hand, this allows an applet – that is, a segment of Java code – to be downloaded from a server, to be run locally by the browser. On the other hand, source code need not be made available to the web. This is a key difference with purely interpreted languages, where source code needs to be furnished.

For application development, Nair et al. [30] list the advantages of using Java for web-based simulations:

- Java has built-in support for producing sophisticated animations.
- Java has built-in threads, asynchronous control mechanisms for logical processes, facilitating the implementation of the process/resource interaction worldview.
- Models implemented as Java applets, can be made widely accessible through web browsers such as Microsoft's Internet Explorer or Netscape's Navigator.
- Java's universal portability eliminates the need to port to a different platform, to recompile, or to relink.

For application execution, both the web and Java offer a set of capabilities to facilitate web-based simulation. Key features of web technology include [15]:

- Transparency of network heterogeneity: Interoperability of different networks is achieved through well-defined, standardized protocols such as HTTP and CGI. The HTTP protocol defines a uniform information transport mechanism. CGI establishes the interface between a web server and arbitrary programs executing on the same machine.
- Transparency of operating system heterogeneity: The Java virtual machine, i.e., a platform-neutral architecture definition, provides a uniform processing environment due to its integration into standard web browsers.
- Transparency of user interface heterogeneity: Along with Java, a class library for user interface programming now exists to support standardized concepts for graphical interfaces. Dynamic retrieval of classes from the net and run-time linking significantly enriches user interfaces.

Fishwick [14] cites at least two ways of combining the web with simulation: (i) distributed model repositories also referred to as Web-enabled Simulation Environ-

ments (WSE), and (ii) parallel and distributed execution also referred to as Web-enabled Distributed Interactive Simulations (WDIS). It therefore comes as no surprise that most of the effort over the past few years has been devoted to the construction of Java-based simulation languages and web-based modeling libraries. There exist a large number of Java-based simulation software packages; some of them are directly available on the Internet. These packages can be categorized into two broad groups: Java implementation of existing simulation packages (e.g., JavaGPSS [25] and Simjava [28]) and new simulation languages implemented in Java (e.g., Simkit [5], JSIM [30], and Silk [24]]. A comprehensive list of Java-based simulation packages can be found at ms.ie.org/websim/survey/survey.html. There is also on-going effort to extend some of these software packages with distributed computing capabilities [32].

Web-based simulation optimization applications, however, have been scarce. To date, we are aware of only the Java-based simulation manager by Marr et al. [27]. In our work, we deploy OCBA to distribute the simulation experiments over the web. This contribution to WSE is described next.

3. Web-based simulation optimization

3.1. Description

As an illustration of simulation optimization in a web-enabled simulation environment, our first prototype is accessed through a graphical user interface using a web browser. The interface is connected to two databases. One database contains different optimization algorithms; the other contains a library of models. The interface, optimization algorithms, and all the simulation models are written in Java. Through the interface, the user can select a simulation model, and specify the parameters for all competing designs. The type of optimization as well as the desired confidence level is also specified. Upon the execution of the experiment, the results are displayed. The current implementation is not capable of storing any of the output results for further – possibly off-line – processing. Nair et al. [30] developed the Query Driven Simulation framework (QDF), which includes a database management system as part of the overall simulation system. Since most browsers restrict Java applets from writing into a file, the QDF approach bypasses this restriction by writing into a database, hence making simulation input and output data available to the user.

Fig. 1 illustrates the basic architecture of our web-based simulation optimization system. Fig. 2 shows the graphical user interface of the current implementation. The latter can be accessed by pointing a browser at mason.gmu.edu/∼cchen9. The current implementation of the web-based simulation optimization system features OCBA as the only optimization tool. The model libraries contain two queueing system models: The first model depicts a single-server queue; the second is a two-station tandem queue.

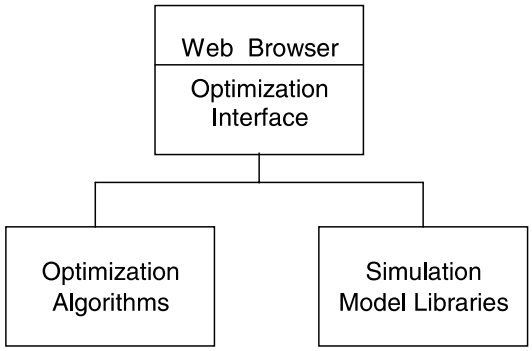

Fig. 1. The structure of the web-based simulation optimization system.

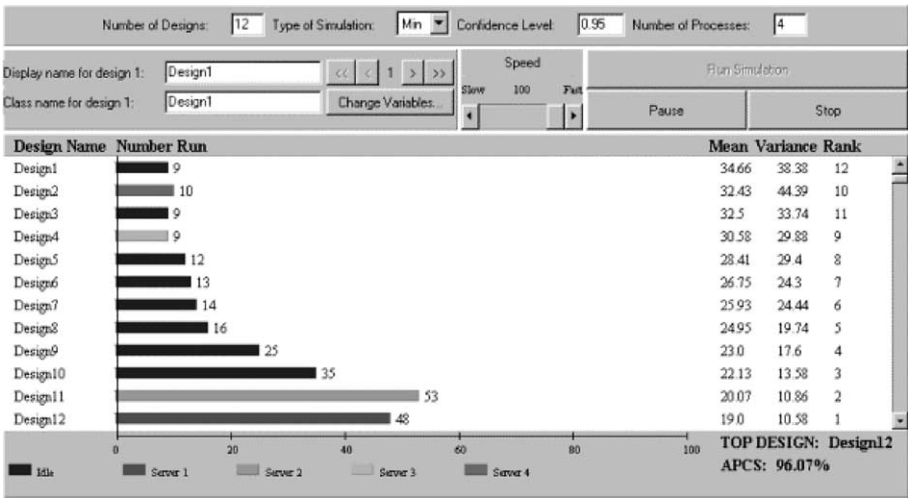

Fig. 2. Web-based simulation optimization interface.

3.2. Illustrative examples

To illustrate the power of the web technology for simulation analysis, we have conducted various experiments applying OCBA to the queueing models mentioned in Section 3.1. We consider a queueing system where the customer interarrival times are independent and distributed uniformly between 0.15 and 0.20 h. The service times are also independent, but exponentially distributed with rate $(6.0 + 0.2i)$ for the ith design. We wish to identify the system that yields the lowest expected system time for the first 100 customers (terminating simulation). The OCBA algorithm is applied to this ranking and selection problem, where the budget allocation is done in a greedy fashion; that is, additional computing budget is allocated to the top m designs that maximize the probability of correct selection, where m is the number of threads

executed concurrently (refer to Section 2.1 for the algorithm expressed in pseudo-code format).

Table 1 summarizes the parameters of the experiment. These parameters are chosen so as to investigate various aspects of the web-based simulation environment; in particular, we consider the complexity of the simulated models, the difficulty of the decision problem expressed as the number of alternative designs, and the computing platform expressed as the hardware, the software, and the browser.

Each parameter combination is investigated through 10 independently seeded replications. The results shown in this section reflect averages across these replications. Fig. 3 depicts the run time performance of the algorithm applied to the single-server queueing model with a 99% probability of correct selection. The notation "IE400,10" denotes an experiment conducted on a 400 MHz processor using the *Microsoft Internet Explorer* browser to select the best out of 10 competing designs. Similarly, "N100,5" denotes an experiment conducted on a 100 MHz processor using the *Netscape Navigator* browser to select the best out of 5 competing designs.

Java's primary advantage is its portability across different platforms. Run-time performance, however, is significantly affected by the particular platform on which

Table 1
Parameters investigated in the experiment

Environment	Parameter	Value
Simulation models	Queueing systems	Single-server model Two-station tandem model
Decision problem	No. of alternative designs	$k = 5$ $k = 10$
Computing platform	Java program	No. of threads, $m = 1, 2, 3$
	Hardware	Pentium II 100 MHz Pentium II 400 MHz
	Browser	Netscape Navigator Internet Explorer

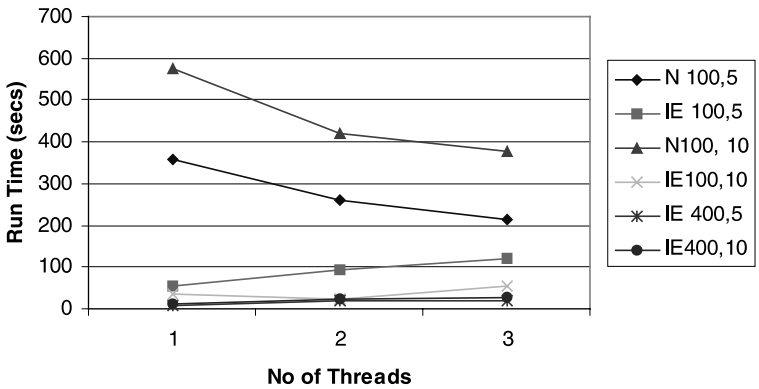

Fig. 3. OCBA run time on various platforms, single-server queueing model, 5 designs.

the experiment is conducted. The impact of the hardware platform is not surprising: the experiment runs faster on a faster processor. However, we have observed differences between the two browsers used on the same problem, irrespective of the hardware platform: The interpreters provided by different browsers handle the byte code generated by Java differently. The differences are even more pronounced in environments that include sophisticated animations and richer graphical displays. These differences tend to be less significant on faster processors.

Java's multithreading capability provides a natural environment for conducting parallel experiments. Fig. 4 summarizes the results from the experiment involving the single-server queueing model with five competing designs, where the desired probability of correct selection is set at 99%. The experiment is conducted on a 400 MHz processor with the IE browser. Different numbers of threads are used. Given the greedy run allocation policy implemented in the OCBA algorithm, a larger number of threads benefits those designs that are initially thought of as inferior or non-promising designs. In the experiment with a single thread, such designs are quickly eliminated from further consideration; however, as the number of threads is increased, they are examined further. This, in turn, increases the average total number of replications conducted in an experiment, as depicted in Fig. 5. For any model and any number of alternative designs considered in our experiments, the average total number of replications increases as we increase the number of threads, giving the inferior or non-promising designs further opportunity to stay in the running. This, in turn, increases the estimated probability of correct selection slightly above the desired value, which is fixed at the start of the experiment.

The total number of replications executed per thread, however, consistently decreases as we increase the number of threads. Fig. 6 shows this phenomenon in an experiment run on a 400 MHz processor with the IE browser, and a probability of correct selection of 99%.

Our experiments point at the tremendous potential for parallel and distributed simulation experiments, where each run is not only distributed across threads but also across different host computers. This is further explored in the next section.

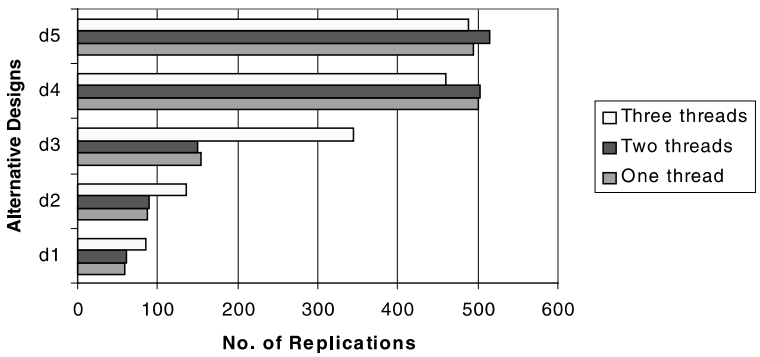

Fig. 4. Distribution of replications among competing designs, single-server queueing model, 5 designs.

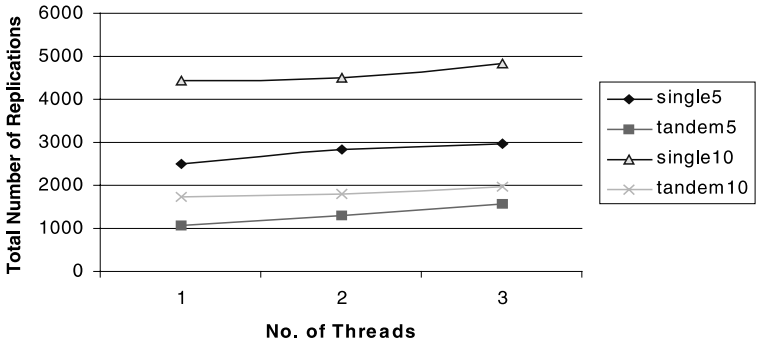

Fig. 5. Impact of multithreading.

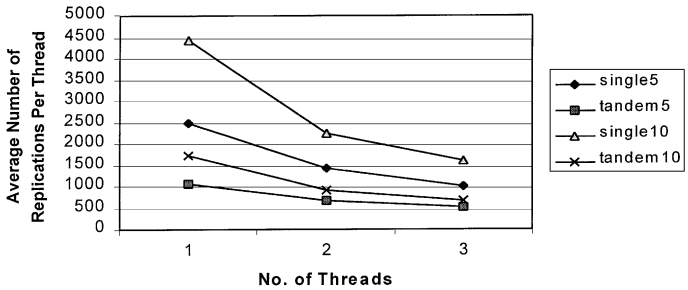

Fig. 6. Average number of replications per thread on a 400 MHz processor with the IE browser.

4. Extension to parallel computation environments

4.1. Description

The key contribution of the OCBA algorithm is the considerable gain in simulation efficiency through significant reduction in computational effort (i.e., the total number of simulation replications) needed to identify the best system or obtain a desired simulation quality. The experiments in Section 3 show that OCBA further offers a natural way of running the optimization in a distributed fashion, thereby providing further gains in simulation efficiency. In our first prototype of the web-based simulation optimization system, we have used Java's threading capability to mimic the parallel simulation environment. While the simulation is actually run on a traditional sequential computer, as illustrated in Fig. 6, the average number of replications executed per thread significantly decreases as we increase the number of threads. This means that the total computation time will decrease as we increase the number of parallel processors, if the OCBA algorithm were implemented in a *real* parallel computation environment and if the network congestion were not an issue. Based on this idea, we developed a real distributed web-based computation environ-

ment with the OCBA algorithm managing the entire simulation experiment. Fig. 7 depicts the architecture of the distributed implementation.

The major components of our distributed implementation for web-based simulation experiments include the following:

Management console: Through a web browser, a central management console is established. The console is used to set up the simulation experiment. In particular, the simulation model is selected, and the corresponding parameters for the design problem are entered for the experiment. As the experiment proceeds, the status of each client (which is defined below) and the most recent simulation results are displayed here (as shown in Fig. 8). The local clients are also displayed on the management console together with information on the particular experiment the client is currently executing. In addition to real-time monitoring of clients, the management console can also be used for post-processing of the simulation results and for the launching of multiple simulation experiments.

Web server and central controller: The Java source code resides in the web server. The OCBA algorithm is executed on this server. Based on the simulated results obtained from each client, OCBA serves as a central controller, and determines which design should be simulated further so that the overall efficiency can be maximized. The central controller then assigns the computational task to a particular client.

Local client: Local clients are viewed as computing resources seeking work. To establish a local client for our web-based distributed simulation, it suffices to point the web browser (on a local computer) to the web server. The status of this client is then displayed, both at the local machine and on the management console. OCBA at the web server will ask the local client to simulate a particular design for a certain number of replications. After the requested simulation is finished, the local client sends the simulated results to the central controller whereupon this client becomes available for new tasks. Based on the simulated results, the central controller determines through OCBA the new task for this client. The parallel computation on local clients can be in synchronous or asynchronous mode.

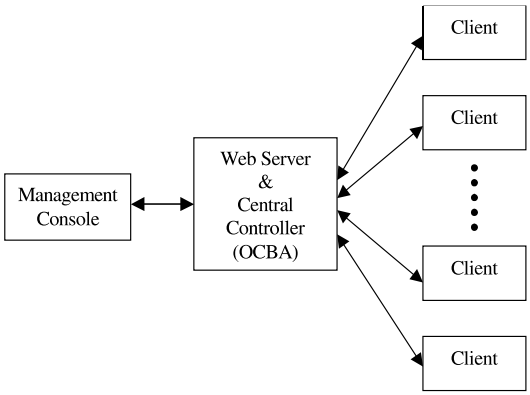

Fig. 7. The structure of the distributed web-based simulation system.

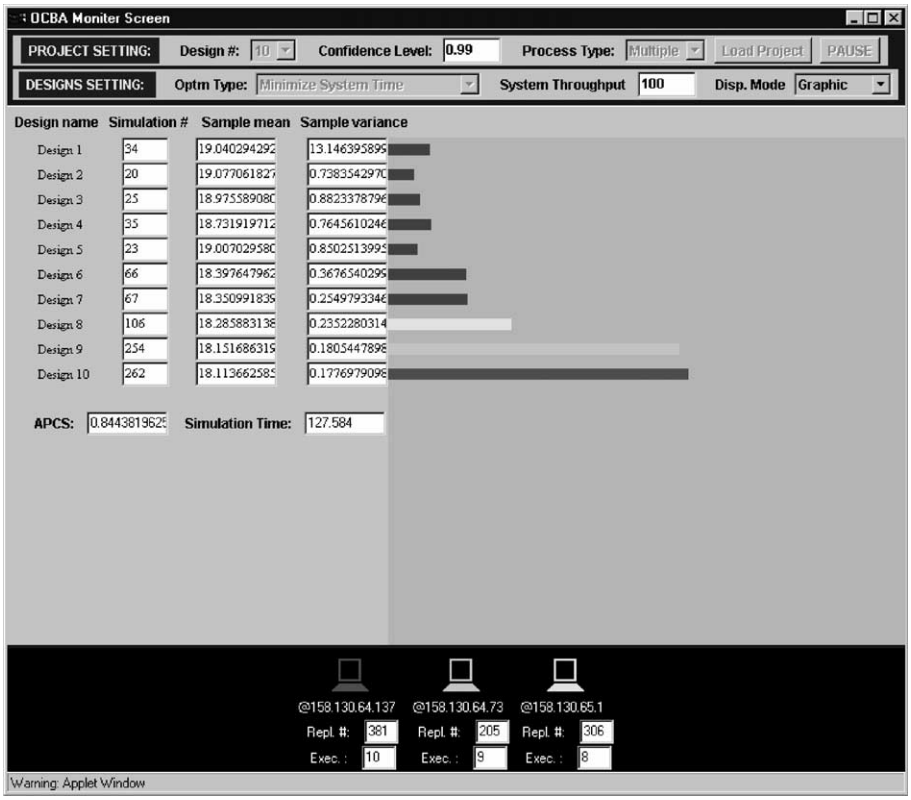

Fig. 8. Management console for the distributed web-based simulation.

4.2. Illustrative examples

Since OCBA has distributed the simulation load among many different computers through the Internet, the simulation time can be dramatically reduced. We perform the same simulation experiment presented in Section 3.2 on the distributed web-based simulation environment: the two-station tandem model with 10 design alternatives using Internet Explorer.

We have two objectives with this test: (1) to demonstrate that the simulation time can be reduced through distributed simulation, and (2) to study the impact of the delay caused by the network communication. To this end, we compare four different settings:

1. *Local Simulation* (LS): All simulations and the OCBA algorithm are performed on a single computer. Namely, the web browser for the local client resides on the same machine as that of the central controller. In this setting, there is no communication through the network, so there is no communication delay. This is the same as the setting in previous experiments; it therefore serves as a benchmark.

2. *Distributed Simulation with a single client* (DS1): There is a single client where the simulations are performed. The difference between DS1 and LS is that in DS1 the client resides on a different machine. Since the OCBA algorithm is executed at the web server, communication between server and client is needed. While the total computation loads in DS1 and LS are almost the same, we expect some communication delay in DS1. Therefore, the total simulation time for DS1 is expected to be somewhat longer than that in LS, although the total number of replications should remain the same.
3. *Distributed Simulation with two clients* (DS2): There are two local clients connected to the web server. Again, the OCBA algorithm is executed at the server, and requests are sent to clients to perform simulations. Simulation results are sent back to the web server, whereupon OCBA determines the new task for clients. Hence, some communication is needed. So, the advantage of distributed simulation may be reduced by the communication delay.
4. *Distributed Simulation with three clients* (DS3): The setting is the same as in DS2 except there are three local clients connected to the web server. In this setting, we anticipate the advantage of distributed simulation to become more significant.

Fig. 9 compares the actual average simulation times for these four different settings. The total number of replications is equivalent to the total simulation load. On the other hand, the total simulation time decreases as we have more local clients. Whereas the simulation loads for LS and DS1 are almost the same, DS1 takes much longer to complete all simulation replications due to communication delays through the Internet in DS1. When the number of clients is increased, the time savings of distributed simulation become more and more significant.

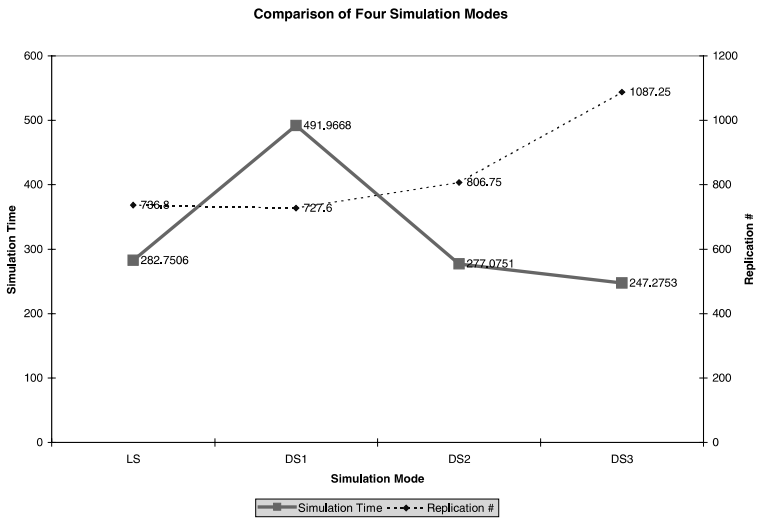

Fig. 9. Total simulation time and number of replications with different number of local clients.

5. Summary and current work

In this paper we introduced a framework for combining the statistical efficiency of simulation optimization techniques with the effectiveness of parallel execution algorithms. In particular, a novel simulation sampling procedure, namely the OCBA algorithm, is implemented in a web-based environment for low-cost parallel and distributed simulation experimentation. We therefore demonstrated the power of web-enabled simulation environments.

More specifically, our work focuses on the parallel execution of a simulation experiment for optimization. However, instead of equally distributing simulation replications for different design alternatives, OCBA offers an intelligent way to allocate experiments to processors. The key contribution of OCBA is the significant gain in simulation efficiency through significant reduction in computational effort (i.e., the total number of simulation replications) needed to identify the best system or obtain a desired simulation quality. Moreover, OCBA offers a natural way of running the optimization in a distributed fashion, thereby providing further gains in simulation efficiency.

The application of parallel simulation technology has traditionally been limited. First, parallel hardware configurations are expensive, hence not widely available to many users. Second, system software to support large-scale distributed simulations remains scarce. Under these circumstances, the simulation community has been reluctant to explore the potential gains offered by this technology. Now, however, the Internet and web-based technologies now provide a viable infrastructure for parallel discrete-event simulation. The Internet eliminates the need for expensive parallel hardware, while the Internet Protocol (IP) unifies diverse networking technologies and administrative domains. In other words, IP ensures uniform connectivity without requiring uniform behavior. The Internet can therefore be viewed as an appealing infrastructure for distributed computation [31].

Our objective in designing and implementing a web-based simulation optimization system is to emphasize the power of web technologies for the experimental design and output analysis phases of a simulation study. Our immediate concern is thus functionality rather than run-time performance. Issues of portability, maintainability, and conformance to standards are crucial in demonstrating the feasibility of a web-based optimization system. We are currently focusing on enhancing the execution speed through the design of intelligent distribution algorithms that exploit emerging technologies aimed at speeding up communication over the Internet. For instance, OCBA is driven solely by statistical efficiency, implicitly assuming that all processors are identical in computing power. A natural extension of the current algorithm explicitly considers the computing power of individual processors when allocating simulation tasks to local clients. In addition to processor capability, communication overhead (e.g., congestion) is also studied, to assess whether the batching of simulation replications may lead to further run-time reductions. A final effort focuses on dealing with unreliable clients that may slow down the overall completion of the simulation experiment, or with unreliable communication networks that may lead to data loss. In summary, we are cur-

rently working on making the web-based distributed simulation environment robust and efficient.

Acknowledgements

The authors are grateful to the anonymous referee whose meticulous reading of the manuscript has led to an improved paper.

References

[1] R.E. Bechhofer, T.J. Santner, D.M. Goldsman, Design and Analysis of Experiments for Statistical Selection, Screening, and Multiple Comparisons, Wiley, New York, 1995.
[2] J.O. Berger, Statistical Decision Theory, Foundations, Concepts, and Methods, Springer, Berlin, 1980.
[3] J.O. Berger, J. Deely, A Bayesian approach to ranking and selection of related means with alternative to AOV methodology, Journal of American Statistics Association 83 (1994) 364–373.
[4] J.M. Bernardo, A.F.M. Smith, Bayesian Theory, Wiley, New York, 1994.
[5] A.H. Buss, K.A. Stork, Discrete event simulation on the World Wide Web using Java, in: Proceedings of the Winter Simulation Conference, 1996, pp. 780–786.
[6] K.M. Chandy, J. Misra, Distributed simulation: a case study in design and verification of distributed programs, IEEE Transactions on Software Engineering SE-5.5 (1979) 440–452.
[7] C.H. Chen, H.C. Chen, L. Dai, A gradient approach of smartly allocating computing budget for discrete event simulation, in: Proceedings of the Winter Simulation Conference, 1996, pp. 398–405.
[8] S.E. Chick, Bayesian analysis for simulation input and output, in: Proceedings of the 1997 Winter Simulation Conference, 1997, pp. 253–260.
[9] W.K. Chiu, The ranking of means of normal populations for a generalized selection goal, Biometrika 61 (1974) 579–584.
[10] J.C. Comfort, The simulation of a master–slave event set processor, Simulation 42.3 (1984) 117–124.
[11] A.I. Concepcion, A hierarchical computer architecture for distributed simulation, IEEE Transactions on Computing C-38.2 (1989) 311–319.
[12] E.J. Dudewicz, S.R. Dalal, Allocation of observations in ranking and selection with unequal variances, Sankhya B 37 (1975) 28–78.
[13] The Economist, Spare Computing Capacity Scattered around a Zillion Desktops Could Soon be Worth Real Money, July 27, 2000.
[14] P.A. Fishwick, Web-based simulation, in: Proceedings of the Winter Simulation Conference, 1997, pp. 100–102.
[15] A. Ferscha, M. Richter, Java based conservative distributed simulation, in: Proceedings of the Winter Simulation Conference, 1997, pp. 381–388.
[16] R,M. Fujimoto, Parallel and distributed simulation, in: Proceedings of the Winter Simulation Conference, 1995, pp. 118–125.
[17] R.M. Fujimoto, Parallel discrete event simulation: will the field survive?, ORSA Journal on Computing 5 (1993) 218–230.
[18] S.S. Gupta, S. Panchapakesan, Multiple Decision Procedures: Theory and Methodology of Selecting and Ranking Populations, Wiley, New York, 1979.
[19] S.S. Gupta, J.O. Berger, Statistical Decision Theory and Related Topics IV, Springer, Berlin, 1988.
[20] P. Heidelberger, Discrete event simulations and parallel processing: statistical properties, SIAM Journal Scientific and Statistical Computing 9.6 (1988) 1114–1132.
[21] J.C. Hsu, Multiple Comparisons Theory and Methods, Chapman & Hall, London, 1996.

[22] I. Kang, I. Lee, State minimization for concurrent system analysis based on state space exploration, in: Proceedings of the Conference on Computer Assurance, 1994.
[23] I. Kang, I. Lee, Efficient state space generation for analysis of real-time systems, in: Proceedings of the ACM International Symposium on Software Testing and Analysis, 1996.
[24] R.A. Kilgore, E. Burke, Object-oriented simulation of distributed systems using Java and Silk, in: Proceedings of the Winter Simulation Conference, 2000, pp. 1802–1809.
[25] U. Klein, S. Strassburger, J. Beikirch, Distributed simulation with JavaGPSS based on the high level architecture, Extended abstract, 1997.
[26] Y.-B. Lin, Parallelism analyzers for parallel discrete event simulation, ACM Transactions on Modeling and Computer Simulation 2.3 (1992) 239–264.
[27] C. Marr, C. Storey, W.E. Biles, J.P.C. Kleijnen, A Java-based simulation manager for Web-based simulation, in: Proceedings of the Winter Simulation Conference, 2000, pp. 1815–1822.
[28] R. McNab, F.W. Howell, Using Java for discrete event simulation, in: Proceedings of the Twelfth UK Computer and Telecommunications Performance Engineering Workshop (UKPEW), University of Edinburgh, 1996, pp. 219–228.
[29] J. Misra, Distributed discrete-event simulation, Computing Surveys 18.1 (1986) 39–65.
[30] R.S. Nair, J.A. Miller, Z. Zhang, Java-based query driven simulation environment, in: Proceedings of the Winter Simulation Conference, 1996, pp. 786–793.
[31] New York Times, Buddy, Can You Spare Some Processing Time? September 1, 2000.
[32] E.H. Page, R.L. Moose Jr., S.P. Griffin, Web-based simulation in SimJava using remote method invocation, in: Proceedings of the Winter Simulation Conference, 1997, pp. 468–474.
[33] V. Paxson, S. Floyd, Why we don't know how to simulate the Internet, in: Proceedings of the Winter Simulation Conference, 1997, pp. 1037–1044.
[34] Y. Rinott, On two-stage selection procedures and related probability inequalities, Communications in Statistics A 7 (1978) 799–811.
[35] T.K. Som, R.G. Sargent, A formal development of event graphs as an aid to structured and efficient simulation programs, ORSA Journal on Computing 1.2 (1989) 107–125.
[36] J.R. Wilson, Variance reduction techniques for digital simulation, American Journal of Mathematical and Management Sciences 4 (1984) 277–312.
[37] E. Yücesan, L. Schruben, Modeling paradigms for discrete event simulations, Operations Research Letters 13.5 (1993) 265–276.